

Содержание

1. Основная терминология удалённых баз данных. Моделирование информационных систем. Этапы проектирования информационной системы	3
2. Назначение и особенности инструментальных средств по созданию баз данных. СУБД PostgreSQL. Характеристики, особенности работы. Основные объекты сервера баз данных	4
3. Стандартные типы данных в СУБД PostgreSQL. Создание собственного типа данных. Слабоструктурированные типы данных	5
4. Проектирование структуры базы данных с помощью команд языка SQL. Реализация наследования. Модификация структуры базы данных, запрет на изменение данных в таблицах	7
5. Пользователи и полномочия. Создание роли, привилегии. Настройка ограничений доступа к данным. Пример	9
6. Форматы операторов на изменение и удаление данных. Ввод данных в таблицы на языке SQL. Полный и сокращённый форматы оператора добавления данных. Удаление записей из таблиц БД	11
7. Транзакции, определение и пример. Виды. Понятие целостности. Свойства ACID. Варианты выполнения транзакций согласно стандарту. Восстановление данных	13
8. Простые и сложные запросы на выборку. Используемые операторы, примеры. Псевдонимы полей и таблиц. Подзапрос как источник данных. Функции	15
9. Запросы на группировку. Запросы по дереву запросов. Функции преобразования типов	17
10. Использование регулярных выражений в SQL-запросах	18
11. Аналитические функции. Синтаксис оконной функции. Примеры	19
12. Множественные операции. Общие правила. Примеры	21
13. Виды соединений. Реализация всех видов соединений на примерах	22
14. Подзапросы, виды. Преимущества, правила создания. Использование агрегатных функций	24
15. Представления. Преимущества и недостатки. Модификация представлений. Групповые и материализованные представления	26
16. Индексы. Создание и применение индексов. Оптимизация запросов. Построение плана запроса. Этапы выполнения запроса	28
17. Методы контроля качества данных. Партиционирование. Примеры	30
18. Язык PL/pgSQL. Анонимные блоки PL/pgSQL. Структура программы, базовые команды. Обработка ошибок	32

19. Тип запись, привязка. Курсоры PL/pgSQL. Примеры	34
20. Хранимые процедуры. Синтаксис создания. Примеры	36
21. Хранимые функции. Синтаксис создания. Отличия от процедур. Табличные функции. Примеры	38
22. Триггеры. Синтаксис создания. Виды и применение. Триггеры событий. Примеры	40
23. Шифрование и хеширование данных	42
24. Угрозы, специфичные для СУБД. Основные принципы безопасности. Персональные данные и конфиденциальность	43
25. Анализ данных средствами SQL: подготовка данных, выявление аномалий, уменьшение размерности	45
26. Анализ данных средствами SQL: временные ряды, когортный анализ, текстовый анализ, анализ экспериментов	47
27. Создание сложных наборов данных. Общие табличные выражения. Расширения для группировки	49
28. Администрирование базы данных: конфигурирование сервера, архитектура, организация данных	51
29. Администрирование базы данных: резервное копирование. Репликация ..	53
30. Администрирование базы данных: мониторинг сервера БД, журналирование, блокировки, задачи администрирования	54

1. Основная терминология удалённых баз данных.

Моделирование информационных систем. Этапы проектирования информационной системы

Удалённая база данных — это база данных, расположенная не на компьютере пользователя, а на отдельном сервере. Клиентское приложение обращается к ней по сети. В такой схеме обычно выделяют клиент, сервер баз данных, сетевой протокол, пользователя, схему данных, таблицы, запросы, транзакции и права доступа. PostgreSQL часто используется именно как серверная СУБД: приложение подключается к серверу, отправляет SQL-запросы, а сервер выполняет их и возвращает результат.

Важные термины: сервер БД — программа, которая хранит и обрабатывает данные; клиент — программа для обращения к серверу, например pgAdmin 4 или приложение; соединение — сетевой сеанс работы с сервером; транзакция — логическая группа операций; схема — пространство имён внутри базы данных; роль — пользователь или группа пользователей с правами доступа.

Моделирование информационной системы нужно, чтобы заранее описать данные, процессы и связи между ними. Обычно используют ER-диаграммы, DFD, IDEF0, UML-диаграммы и словесное описание предметной области. Например, для автошколы можно выделить сущности «ученик», «инструктор», «группа», «занятие», «оплата».

Основные этапы проектирования информационной системы: анализ предметной области, выделение требований, построение инфологической модели, создание логической модели БД, физическое проектирование, реализация в СУБД, тестирование, настройка прав доступа и сопровождение. На практике сначала нужно понять, какие данные хранятся и какие операции выполняют пользователи, а уже потом писать SQL.

Пример простой таблицы, которая появляется после анализа предметной области:

```
create table students (  
    id bigserial primary key,  
    full_name text not null,  
    phone text,  
    created_at timestamp default now()  
);
```

2. Назначение и особенности инструментальных средств по созданию баз данных. СУБД PostgreSQL. Характеристики, особенности работы. Основные объекты сервера баз данных

Инструментальные средства для создания баз данных помогают проектировать структуру, писать SQL-запросы, управлять пользователями, проверять данные, делать резервные копии и анализировать производительность. К таким средствам относятся pgAdmin 4, psql, DBeaver, DataGrip, ERD-инструменты и средства администрирования сервера.

pgAdmin 4 — графический инструмент для работы с PostgreSQL. В нём можно создавать базы данных, схемы, таблицы, пользователей, писать SQL-запросы, смотреть планы выполнения запросов, выполнять резервное копирование и восстановление. psql — консольный клиент, который часто удобнее для администрирования и автоматизации.

PostgreSQL — объектно-реляционная СУБД с открытым исходным кодом. Она поддерживает SQL, транзакции, ограничения целостности, индексы, представления, материализованные представления, процедуры, функции, триггеры, наследование, JSON/JSONB, расширения и гибкую систему ролей. Важная особенность PostgreSQL — расширяемость: можно создавать собственные типы, функции, операторы и подключать расширения.

Основные объекты сервера PostgreSQL: кластер, база данных, схема, таблица, представление, индекс, последовательность, тип данных, функция, процедура, триггер, роль, табличное пространство и расширение. Кластер PostgreSQL может содержать несколько баз данных, каждая база данных может содержать несколько схем, а внутри схем находятся таблицы и другие объекты.

Пример создания базы данных и схемы:

```
create database driving_school;  
  
create schema app;  
  
create table app.clients (  
    id bigserial primary key,  
    full_name text not null  
);
```

3. Стандартные типы данных в СУБД PostgreSQL.

Создание собственного типа данных.

Слабоструктурированные типы данных

Тип данных определяет, какие значения можно хранить в поле таблицы и какие операции над ними доступны. В PostgreSQL есть числовые типы, строковые типы, дата и время, логический тип, перечисления, массивы, UUID, JSON/JSONB и другие.

К числовым типам относятся `smallint`, `integer`, `bigint`, `numeric`, `real` и `double precision`. Для строк используются `char`, `varchar` и `text`. Для даты и времени применяются `date`, `time`, `timestamp`, `timestampz` и `interval`. Логический тип `boolean` хранит `true` или `false`. Для денежных и точных расчётов лучше использовать `numeric`, а не `real`, потому что `real` и `double precision` являются приближёнными типами.

Собственный тип данных можно создать, например, как перечисление. Это удобно, когда поле может принимать только заранее известные значения: статус заказа, роль пользователя, тип занятия.

```
create type lesson_status as enum ('planned', 'completed',  
    'cancelled');  
  
create table lessons (  
    id bigserial primary key,  
    topic text not null,  
    status lesson_status not null default 'planned'  
);
```

Слабоструктурированные типы данных применяются, когда структура данных может меняться или заранее неизвестна. В PostgreSQL для этого часто используют `json` и `jsonb`. Тип `json` хранит текстовое представление JSON, а `jsonb` хранит данные в бинарном виде и обычно удобнее для поиска и индексации.

```
create table event_log (  
    id bigserial primary key,  
    event_name text not null,  
    payload jsonb not null,  
    created_at timestamp default now()  
);  
  
insert into event_log (event_name, payload)
```

```
values ('login', '{"user_id": 10, "ip": "127.0.0.1"}');
```

```
select payload ->> 'ip' as ip_address  
from event_log;
```

4. Проектирование структуры базы данных с помощью команд языка SQL. Реализация наследования. Модификация структуры базы данных, запрет на изменение данных в таблицах

Структура базы данных создаётся с помощью команд DDL: create, alter, drop и truncate. Команда create создаёт объект, alter изменяет его структуру, drop удаляет объект, truncate быстро очищает таблицу. При проектировании важно задавать первичные ключи, внешние ключи, ограничения not null, unique и check, потому что они защищают данные от некорректных значений.

Пример создания связанных таблиц:

```
create table instructors (  
    id bigserial primary key,  
    full_name text not null  
);  
  
create table lessons (  
    id bigserial primary key,  
    instructor_id bigint not null references instructors(id),  
    lesson_date date not null,  
    topic text not null  
);
```

Наследование в PostgreSQL позволяет одной таблице наследовать столбцы другой таблицы. Это редко используют в обычных проектах, потому что чаще применяют связи через внешние ключи или партиционирование. Однако наследование может быть полезно, если несколько таблиц имеют общую структуру.

```
create table documents (  
    id bigserial primary key,  
    title text not null,  
    created_at timestamp default now()  
);  
  
create table contracts (  
    client_name text not null,  
    amount numeric(12,2) not null  
) inherits (documents);
```

Модификация структуры выполняется через `alter table`. Можно добавить столбец, изменить тип, добавить ограничение или удалить поле.

```
alter table lessons add column room text;  
  
alter table lessons  
add constraint lesson_date_check check (lesson_date >= date  
'2024-01-01');
```

Запретить изменение данных можно с помощью прав доступа, триггеров или правил. Наиболее простой способ — отозвать права `insert`, `update` и `delete` у пользователя.

```
revoke insert, update, delete on lessons from app_user;  
grant select on lessons to app_user;
```


5. Пользователи и полномочия. Создание роли, привилегии. Настройка ограничений доступа к данным. Пример

В PostgreSQL пользователи и группы называются ролями. Роль может иметь право входа на сервер или быть только групповой ролью. Права доступа определяют, какие действия пользователь может выполнять: читать данные, добавлять записи, изменять строки, удалять данные, создавать объекты и выполнять функции.

Роль с параметром `login` может подключаться к серверу. Роль без `login` обычно используют как группу прав. Это удобно: сначала создаётся групповая роль, ей выдаются права, а затем конкретные пользователи включаются в эту роль.

```
create role app_readonly;  
  
create role ivan login password 'strong_password';  
  
grant app_readonly to ivan;
```

Привилегии выдаются командой `grant` и отзываются командой `revoke`. Для таблиц часто используют права `select`, `insert`, `update` и `delete`. Для схемы нужно отдельно разрешить `usage`, иначе пользователь не сможет обращаться к объектам внутри схемы.

```
grant usage on schema app to app_readonly;  
grant select on all tables in schema app to app_readonly;  
  
revoke delete on all tables in schema app from app_readonly;
```

Пример ограничения доступа: секретарь автошколы может видеть клиентов и группы, но не может удалять оплаты.

```
create role secretary login password 'secretary_password';  
  
grant usage on schema app to secretary;  
grant select, insert, update on app.clients to secretary;  
grant select on app.payments to secretary;  
revoke delete on app.payments from secretary;
```

Для более тонкой настройки можно использовать `row level security`, то есть безопасность на уровне строк. Например, пользователь видит только свои

записи. Это полезно в многопользовательских системах, но требует аккуратной настройки политик.

6. Форматы операторов на изменение и удаление данных. Ввод данных в таблицы на языке SQL. Полный и сокращённый форматы оператора добавления данных. Удаление записей из таблиц БД

Для изменения данных используются операторы insert, update и delete. Insert добавляет строки, update изменяет существующие строки, delete удаляет строки. Эти команды относятся к DML, то есть к языку манипулирования данными.

Полный формат insert явно указывает список столбцов. Это наиболее безопасный и читаемый вариант, потому что порядок значений не зависит от порядка столбцов в таблице.

```
insert into clients (full_name, phone)
values ('Иванов Иван', '+79990000000');
```

Сокращённый формат insert не указывает столбцы. Его лучше использовать только в учебных примерах, потому что при изменении структуры таблицы запрос может сломаться.

```
insert into clients
values (1, 'Петров Пётр', '+79991111111');
```

Можно добавить сразу несколько строк:

```
insert into clients (full_name, phone)
values
    ('Сидорова Анна', '+79992222222'),
    ('Кузнецов Олег', '+79993333333');
```

Оператор update изменяет данные по условию. Без where он изменит все строки таблицы, поэтому условие нужно писать внимательно.

```
update clients
set phone = '+79994444444'
where id = 1;
```

Оператор delete удаляет строки. Без where он удалит все записи из таблицы. Если нужно быстро очистить таблицу целиком, используют truncate, но он работает грубее и не всегда подходит при наличии внешних ключей.

```
delete from clients  
where id = 2;
```

Для проверки изменений удобно использовать `returning`. Он возвращает изменённые или удалённые строки.

```
delete from clients  
where id = 3  
returning id, full_name;
```

7. Транзакции, определение и пример. Виды. Понятие целостности. Свойства ACID. Варианты выполнения транзакций согласно стандарту. Восстановление данных

Транзакция — это последовательность операций, которая выполняется как единое целое. Если все операции успешны, изменения фиксируются командой `commit`. Если произошла ошибка, изменения можно отменить командой `rollback`. Транзакции нужны для сохранения целостности данных.

Простой пример: при оплате заказа нужно добавить платёж и изменить статус заказа. Нельзя допустить ситуацию, когда платёж добавился, а статус не изменился из-за ошибки.

```
begin;

insert into payments (order_id, amount, paid_at)
values (10, 5000, now());

update orders
set status = 'paid'
where id = 10;

commit;
```

Если ошибка обнаружена до фиксации, транзакцию можно отменить:

```
begin;

update accounts
set balance = balance - 1000
where id = 1;

rollback;
```

Свойства ACID:

- atomicity — атомарность, то есть транзакция выполняется полностью или не выполняется вовсе;
- consistency — согласованность, то есть после транзакции данные не нарушают ограничения;

- isolation — изолированность, то есть параллельные транзакции не должны некорректно мешать друг другу;
- durability — долговечность, то есть после commit данные должны сохраниться даже при сбое.

Стандарт SQL выделяет уровни изоляции: read uncommitted, read committed, repeatable read и serializable. В PostgreSQL фактически read uncommitted ведёт себя как read committed. Чем выше уровень изоляции, тем меньше аномалий параллельного выполнения, но тем выше накладные расходы.

Восстановление данных обеспечивается журналом предзаписи WAL, резервными копиями и архивацией журналов. Если сервер аварийно завершился, PostgreSQL использует WAL для приведения базы данных в согласованное состояние.

8. Простые и сложные запросы на выборку.

Используемые операторы, примеры. Псевдонимы полей и таблиц. Подзапрос как источник данных. Функции

Оператор `select` используется для выборки данных. Простой запрос получает данные из одной таблицы, а сложный может использовать соединения, подзапросы, группировку, оконные функции и общие табличные выражения.

Пример простого запроса:

```
select id, full_name, phone
from clients
where phone is not null
order by full_name;
```

Псевдонимы делают результат понятнее. Для столбцов применяется `as`, для таблиц часто используют короткие алиасы.

```
select c.full_name as client_name,
       c.phone as client_phone
from clients as c;
```

Сложный запрос может объединять несколько таблиц. Например, можно получить список занятий вместе с фамилией инструктора.

```
select l.lesson_date,
       l.topic,
       i.full_name as instructor_name
from lessons as l
join instructors as i on i.id = l.instructor_id
where l.lesson_date >= current_date
order by l.lesson_date;
```

Подзапрос как источник данных помещается в `from`. Он должен иметь псевдоним.

```
select t.instructor_id, t.lessons_count
from (
    select instructor_id, count(*) as lessons_count
    from lessons
    group by instructor_id
)
```

```
) as t  
where t.lessons_count > 5;
```

В PostgreSQL есть много встроенных функций. Строковые функции позволяют менять регистр, извлекать часть строки, объединять текст. Функции даты и времени позволяют получать текущую дату, извлекать год, месяц или разницу между датами.

```
select upper(full_name) as upper_name,  
       length(full_name) as name_length  
from clients;  
  
select now() as current_time,  
       extract(year from now()) as current_year;
```


9. Запросы на группировку. Запросы по дереву запросов. Функции преобразования типов

Группировка используется, когда нужно посчитать агрегированные значения по группам: количество, сумму, среднее, минимум или максимум. Для этого применяются `group by` и агрегатные функции `count`, `sum`, `avg`, `min`, `max`.

```
select instructor_id,  
       count(*) as lessons_count  
from lessons  
group by instructor_id;
```

Если нужно отфильтровать строки до группировки, используется `where`. Если нужно отфильтровать уже готовые группы, используется `having`.

```
select instructor_id,  
       count(*) as lessons_count  
from lessons  
where lesson_date >= date '2026-01-01'  
group by instructor_id  
having count(*) >= 10;
```

Дерево запроса — это логическая структура выполнения запроса: сначала определяется источник данных, затем условия, соединения, группировка, фильтрация групп, выбор выражений, сортировка и ограничение результата. Важно понимать, что порядок написания SQL не полностью совпадает с логическим порядком обработки. Например, `where` выполняется до `select`, поэтому нельзя напрямую использовать псевдоним из `select` в `where`.

Функции преобразования типов нужны, когда значение нужно привести к другому типу. В PostgreSQL можно использовать `cast` или короткий синтаксис через двойное двоеточие.

```
select cast('2026-05-16' as date) as d1,  
       '2026-05-16'::date as d2,  
       '1500.50'::numeric as amount;
```

Преобразования нужно применять осторожно. Если строка не соответствует нужному формату, PostgreSQL выдаст ошибку. Поэтому при загрузке грязных данных часто сначала используют временные таблицы со строковыми полями, а затем очищают и преобразуют значения.

10. Использование регулярных выражений в SQL-запросах

Регулярные выражения позволяют искать строки по шаблону, а не только по точному совпадению. В PostgreSQL для этого используются операторы `~`, `~*`, `!~` и `!~*`. Оператор `~` проверяет соответствие регулярному выражению с учётом регистра, а `~*` — без учёта регистра. Операторы `!~` и `!~*` означают отрицание.

Например, можно найти клиентов, у которых телефон начинается с российского кода +7:

```
select full_name, phone
from clients
where phone ~ '^\\+7';
```

Можно найти строки, в которых есть слово «тест» без учёта регистра:

```
select full_name
from clients
where full_name ~* 'тест';
```

Регулярные выражения полезны для проверки формата данных. Например, можно найти некорректные email-адреса. Такой шаблон не является идеальной проверкой email, но подходит для базового контроля.

```
select email
from users
where email !~* '^[a-z0-9._%+-]+@[a-z0-9.-]+\\. [a-z]{2,}$';
```

Также PostgreSQL поддерживает функции `regexp_replace`, `regexp_match`, `regexp_matches` и `regexp_split_to_table`. Например, можно очистить телефон от лишних символов и оставить только цифры.

```
select phone,
       regexp_replace(phone, '[^0-9]', '', 'g') as only_digits
from clients;
```

Регулярные выражения удобны, но на больших таблицах могут быть медленными, особенно если шаблон не позволяет эффективно использовать индекс. Для ускорения текстового поиска иногда применяют специальные индексы и расширение `pg_trgm`.

11. Аналитические функции. Синтаксис оконной функции. Примеры

Оконные функции выполняют вычисления по набору строк, связанных с текущей строкой, но не сворачивают результат в одну строку, как `group by`. Благодаря этому можно одновременно видеть исходные строки и аналитические показатели: номер строки, ранг, накопительную сумму, среднее по группе.

Общий синтаксис оконной функции:

```
function_name(arguments) over (  
    partition by column_name  
    order by column_name  
    rows between unbounded preceding and current row  
)
```

`partition by` делит строки на группы, `order by` задаёт порядок внутри окна, а `frame-условие` определяет границы окна. Не все оконные функции требуют все эти части.

Пример: пронумеровать платежи каждого клиента по дате.

```
select client_id,  
       paid_at,  
       amount,  
       row_number() over (  
           partition by client_id  
           order by paid_at  
       ) as payment_number  
from payments;
```

Пример накопительной суммы платежей:

```
select client_id,  
       paid_at,  
       amount,  
       sum(amount) over (  
           partition by client_id  
           order by paid_at  
           rows between unbounded preceding and current row  
       ) as running_total  
from payments;
```

Функции lag и lead позволяют получить предыдущее или следующее значение. Это удобно для анализа временных рядов.

```
select client_id,  
       paid_at,  
       amount,  
       amount - lag(amount) over (  
         partition by client_id  
         order by paid_at  
       ) as diff_from_previous  
from payments;
```

Оконные функции часто применяются в отчётах, рейтингах, финансовом анализе и поиске изменений между соседними событиями.

12. Множественные операции. Общие правила.

Примеры

Множественные операции объединяют результаты нескольких запросов. В SQL используются `union`, `union all`, `intersect` и `except`. Они работают не со строками таблиц напрямую, а с результатами `select`-запросов.

Главное правило: запросы должны возвращать одинаковое количество столбцов, а соответствующие столбцы должны иметь совместимые типы данных.

Названия столбцов обычно берутся из первого запроса.

`union` объединяет результаты и убирает дубликаты. `union all` объединяет результаты без удаления дубликатов, поэтому обычно работает быстрее.

```
select full_name from students
union
select full_name from instructors;
```

```
select full_name from students
union all
select full_name from instructors;
```

`intersect` возвращает строки, которые есть в обоих результатах. Например, можно найти людей, которые одновременно есть в таблице клиентов и сотрудников.

```
select phone from clients
intersect
select phone from employees;
```

`except` возвращает строки из первого результата, которых нет во втором. Например, можно найти клиентов, у которых нет оплат.

```
select id from clients
except
select client_id from payments;
```

Сортировка для всего результата пишется в конце. Если нужно сортировать отдельные части, их обычно помещают в подзапросы. На практике чаще всего применяют `union all`, если не нужно специально удалять дубликаты.

13. Виды соединений. Реализация всех видов соединений на примерах

Соединения нужны, чтобы получать данные из нескольких таблиц. Основные виды соединений: inner join, left join, right join, full join, cross join и self join.

Условие соединения обычно задаётся через on.

inner join возвращает только те строки, для которых нашлась пара в обеих таблицах.

```
select c.full_name, o.id as order_id
from clients as c
inner join orders as o on o.client_id = c.id;
```

left join возвращает все строки из левой таблицы и подходящие строки из правой. Если пары нет, справа будут null. Это удобно для поиска клиентов без заказов.

```
select c.full_name, o.id as order_id
from clients as c
left join orders as o on o.client_id = c.id
where o.id is null;
```

right join похож на left join, но сохраняет все строки из правой таблицы. В реальной работе его часто заменяют на left join, поменяв таблицы местами.

```
select c.full_name, o.id as order_id
from clients as c
right join orders as o on o.client_id = c.id;
```

full join возвращает все строки из обеих таблиц. Если пары нет, недостающая сторона заполняется null.

```
select c.full_name, o.id as order_id
from clients as c
full join orders as o on o.client_id = c.id;
```

cross join создаёт декартово произведение, то есть каждую строку первой таблицы соединяет с каждой строкой второй. Его используют осторожно.

```
select c.full_name, s.name as service_name
from clients as c
cross join services as s;
```

self join — это соединение таблицы самой с собой. Например, если у сотрудника есть руководитель из той же таблицы.

```
select e.full_name as employee,
       m.full_name as manager
from employees as e
left join employees as m on m.id = e.manager_id;
```

14. Подзапросы, виды. Преимущества, правила создания. Использование агрегатных функций

Подзапрос — это запрос, вложенный в другой SQL-запрос. Он может находиться в select, from, where, having или даже в insert/update/delete.

Подзапросы помогают разбивать сложную задачу на более понятные части.

Основные виды подзапросов: скалярный подзапрос, возвращающий одно значение; табличный подзапрос, возвращающий набор строк и столбцов; коррелированный подзапрос, который зависит от строки внешнего запроса.

Скалярный подзапрос можно использовать для сравнения со средним значением:

```
select id, amount
from payments
where amount > (
    select avg(amount)
    from payments
);
```

Подзапрос в from работает как временная таблица и обязательно должен иметь псевдоним.

```
select t.client_id, t.total_amount
from (
    select client_id, sum(amount) as total_amount
    from payments
    group by client_id
) as t
where t.total_amount > 10000;
```

Коррелированный подзапрос выполняется с учётом текущей строки внешнего запроса. Например, можно найти клиентов, у которых есть хотя бы один заказ.

```
select c.full_name
from clients as c
where exists (
    select 1
    from orders as o
    where o.client_id = c.id
);
```


Преимущества подзапросов: читаемость, возможность пошагово выразить логику, удобство с агрегатами. Недостатки: некоторые подзапросы могут быть менее эффективными, чем соединения или CTE. В PostgreSQL оптимизатор часто умеет преобразовывать запросы, но всё равно лучше проверять план выполнения через explain.

15. Представления. Преимущества и недостатки.

Модификация представлений. Групповые и материализованные представления

Представление — это сохранённый SQL-запрос, к которому можно обращаться как к таблице. Оно не хранит данные само по себе, а каждый раз выполняет запрос к исходным таблицам. Представления помогают скрыть сложность запросов, ограничить доступ к отдельным полям и создать удобный слой для отчётов.

```
create view client_orders as
select c.id as client_id,
       c.full_name,
       o.id as order_id,
       o.status,
       o.created_at
from clients as c
join orders as o on o.client_id = c.id;
```

Преимущества представлений: упрощают запросы, повышают читаемость, позволяют скрыть чувствительные столбцы, помогают отделить структуру БД от приложения. Недостатки: сложные представления могут быть медленными, иногда их нельзя изменять напрямую, а чрезмерное количество вложенных представлений усложняет поддержку.

Модификация представления выполняется через `create or replace view`. Полностью удалить представление можно командой `drop view`.

```
create or replace view client_orders as
select c.full_name,
       o.id as order_id,
       o.status
from clients as c
join orders as o on o.client_id = c.id;
```

Групповое представление содержит агрегаты и `group by`. Обычно такие представления используют для отчётов.

```
create view monthly_payments as
select date_trunc('month', paid_at)::date as month,
       sum(amount) as total_amount,
       count(*) as payments_count
```

```
from payments
group by date_trunc('month', paid_at)::date;
```

Материализованное представление хранит результат запроса физически. Оно быстрее при чтении, но данные в нём нужно обновлять вручную.

```
create materialized view mv_monthly_payments as
select date_trunc('month', paid_at)::date as month,
       sum(amount) as total_amount
from payments
group by date_trunc('month', paid_at)::date;

refresh materialized view mv_monthly_payments;
```

16. Индексы. Создание и применение индексов.

Оптимизация запросов. Построение плана запроса. Этапы выполнения запроса

Индекс — это структура данных, которая ускоряет поиск строк в таблице. Его можно сравнить с оглавлением в книге: без индекса серверу часто приходится просматривать всю таблицу, а с индексом он быстрее находит нужные строки. Однако индекс занимает место и замедляет insert, update и delete, потому что его тоже нужно обновлять.

Обычный индекс создаётся командой create index. PostgreSQL автоматически создаёт индекс для primary key и unique.

```
create index idx_clients_phone on clients(phone);

select *
from clients
where phone = '+79990000000';
```

Для составных условий используют составные индексы. Важно учитывать порядок столбцов.

```
create index idx_lessons_instructor_date
on lessons(instructor_id, lesson_date);
```

План запроса показывает, как PostgreSQL собирается выполнить запрос. Его строят через explain, а фактическое выполнение смотрят через explain analyze. В pgAdmin 4 можно открыть query tool, написать запрос и посмотреть план на вкладке Explain.

```
explain analyze
select *
from lessons
where instructor_id = 5
and lesson_date >= date '2026-01-01';
```

Основные этапы выполнения запроса: разбор SQL, проверка прав, построение дерева запроса, оптимизация, выбор плана, выполнение плана и возврат результата. Оптимизатор выбирает между последовательным чтением, индексным чтением, разными алгоритмами соединений и сортировки.

Оптимизация запросов включает создание подходящих индексов, анализ плана, отказ от лишних столбцов, уменьшение объёма обрабатываемых данных, корректные условия фильтрации и обновление статистики через `analyze`.

```
analyze lessons;
```

Индекс не всегда используется. Если запрос возвращает большую часть таблицы, последовательное чтение может быть выгоднее индексного.

17. Методы контроля качества данных.

Партиционирование. Примеры

Качество данных означает, что данные точные, полные, непротиворечивые, актуальные и соответствуют нужным форматам. В базе данных качество обеспечивают ограничения, типы данных, внешние ключи, уникальность, проверки check, триггеры, транзакции и процедуры загрузки данных.

Пример ограничений качества данных:

```
create table payments (  
    id bigserial primary key,  
    order_id bigint not null references orders(id),  
    amount numeric(12,2) not null check (amount > 0),  
    paid_at timestamp not null default now(),  
    receipt_number text unique  
);
```

Также качество контролируют на этапе загрузки: данные можно сначала помещать во временную таблицу, проверять формат, искать пропуски и дубликаты, а затем переносить в основную таблицу.

```
select phone, count(*)  
from clients  
group by phone  
having count(*) > 1;
```

Партиционирование — это разделение большой таблицы на части по определённому правилу. Для пользователя это одна таблица, но физически данные лежат в нескольких партициях. Партиционирование помогает ускорить запросы, упростить удаление старых данных и управлять большими объёмами.

Пример партиционирования по дате:

```
create table payments_log (  
    id bigint not null,  
    paid_at date not null,  
    amount numeric(12,2) not null  
) partition by range (paid_at);  
  
create table payments_log_2026_01 partition of payments_log  
for values from ('2026-01-01') to ('2026-02-01');
```

```
create table payments_log_2026_02 partition of payments_log  
for values from ('2026-02-01') to ('2026-03-01');
```

Если запрос содержит условие по ключу партиционирования, PostgreSQL может читать только нужные партии. Это называется pruning, то есть отсечение лишних разделов.

18. Язык PL/pgSQL. Анонимные блоки PL/pgSQL.

Структура программы, базовые команды.

Обработка ошибок

PL/pgSQL — процедурный язык PostgreSQL. Он позволяет писать функции, процедуры, триггеры и анонимные блоки с переменными, условиями, циклами и обработкой ошибок. SQL хорошо подходит для описания набора данных, а PL/pgSQL удобен для процедурной логики.

Анонимный блок выполняется через `do`. Он не сохраняется в базе данных, а просто выполняет код один раз.

```
do $$
declare
    v_count integer;
begin
    select count(*) into v_count
    from clients;

    raise notice 'Количество клиентов: %', v_count;
end;
$$;
```

Структура PL/pgSQL обычно состоит из `declare` и `begin...end`. В `declare` объявляются переменные, в `begin` выполняются команды. Можно использовать `if`, `case`, `loop`, `while`, `for`, `select into`, `perform` и `raise`.

```
do $$
declare
    v_amount numeric := 1500;
begin
    if v_amount <= 0 then
        raise exception 'Сумма должна быть положительной';
    else
        raise notice 'Сумма корректна';
    end if;
end;
$$;
```

Обработка ошибок выполняется через блок `exception`. Это похоже на `try/catch` в языках программирования.


```
do $$  
begin  
    insert into clients (full_name, phone)  
    values ('Тестовый клиент', '+79990000000');  
exception  
    when unique_violation then  
        raise notice 'Такая запись уже существует';  
    when others then  
        raise notice 'Другая ошибка: %', sqlerrm;  
end;  
$$;
```

PL/pgSQL полезен для бизнес-логики, но не стоит переносить в него весь код приложения. Лучше хранить в БД критичные операции, которые должны выполняться единообразно и защищённо.

19. Тип запись, привязка. Курсоры PL/pgSQL.

Примеры

В PL/pgSQL тип record используется для хранения строки с заранее неизвестной структурой. Это удобно, когда результат запроса может иметь разные наборы столбцов. Также можно объявлять переменную с привязкой к типу строки таблицы через имя_таблицы%rowtype. Тогда переменная будет иметь такую же структуру, как строка таблицы.

Пример record:

```
do $$
declare
    r record;
begin
    for r in
        select id, full_name
        from clients
    loop
        raise notice 'Клиент: %, %', r.id, r.full_name;
    end loop;
end;
$$;
```

Пример привязки к строке таблицы:

```
do $$
declare
    v_client clients%rowtype;
begin
    select * into v_client
    from clients
    where id = 1;

    raise notice 'Имя: %', v_client.full_name;
end;
$$;
```

Привязка к типу столбца выполняется через %type. Это удобно, потому что если тип столбца изменится, переменная автоматически будет соответствовать новому типу.

```

do $$
declare
    v_phone clients.phone%type;
begin
    select phone into v_phone
    from clients
    where id = 1;
end;
$$;

```

Курсор позволяет построчно обрабатывать результат запроса. В обычном SQL лучше работать наборами данных, но курсоры бывают полезны для сложной процедурной обработки.

```

do $$
declare
    cur_clients cursor for
        select id, full_name from clients;
    v_id bigint;
    v_name text;
begin
    open cur_clients;

    loop
        fetch cur_clients into v_id, v_name;
        exit when not found;
        raise notice 'Клиент: %, %', v_id, v_name;
    end loop;

    close cur_clients;
end;
$$;

```

20. Хранимые процедуры. Синтаксис создания.

Примеры

Хранимая процедура — это объект базы данных, содержащий набор команд. В PostgreSQL процедура создаётся командой `create procedure` и вызывается командой `call`. Процедуры удобны для операций, которые изменяют данные и выполняют бизнес-процесс: провести оплату, закрыть заказ, создать отчётную запись.

Синтаксис создания процедуры:

```
create procedure procedure_name(parameters)
language plpgsql
as $$
begin
    -- команды
end;
$$;
```

Пример процедуры для изменения статуса заказа:

```
create procedure change_order_status(
    p_order_id bigint,
    p_status text
)
language plpgsql
as $$
begin
    update orders
    set status = p_status
    where id = p_order_id;

    if not found then
        raise exception 'Заказ с id % не найден', p_order_id;
    end if;
end;
$$;
```

Вызов процедуры:

```
call change_order_status(10, 'completed');
```

Процедуры отличаются от функций тем, что вызываются через `call` и не обязаны возвращать значение. В PostgreSQL процедуры могут управлять транзакциями в

определённых условиях, например выполнять `commit` или `rollback`, если вызов происходит не внутри уже открытой транзакции.

Процедуры хорошо подходят для административных и бизнес-операций, но для вычислений и получения результата чаще используют функции.

21. Хранимые функции. Синтаксис создания.

Отличия от процедур. Табличные функции.

Примеры

Хранимая функция — это объект базы данных, который принимает параметры, выполняет действия и возвращает результат. Функции можно использовать в `select`, `where`, `join` и других частях SQL-запроса. Они создаются командой `create function`.

Пример простой функции:

```
create function get_client_orders_count(p_client_id bigint)
returns integer
language plpgsql
as $$
declare
    v_count integer;
begin
    select count(*) into v_count
    from orders
    where client_id = p_client_id;

    return v_count;
end;
$$;
```

Вызов функции:

```
select get_client_orders_count(5);
```

Главное отличие функции от процедуры: функция возвращает значение и может использоваться внутри SQL-запроса, а процедура вызывается отдельно через `call` и обычно описывает действие. Функции не стоит использовать для скрытого изменения данных внутри обычной выборки, потому что это усложняет понимание логики.

Табличная функция возвращает набор строк. Это удобно для отчётов.

```
create function get_month_payments(p_year integer, p_month
integer)
returns table (
    payment_day date,
    payments_count bigint,
```

```
        total_amount numeric
    )
language sql
as $$
    select paid_at::date as payment_day,
           count(*) as payments_count,
           sum(amount) as total_amount
    from payments
    where extract(year from paid_at) = p_year
          and extract(month from paid_at) = p_month
    group by paid_at::date
    order by payment_day;
$$;
```

Вызов табличной функции:

```
select *
from get_month_payments(2026, 5);
```

22. Триггеры. Синтаксис создания. Виды и применение. Триггеры событий. Примеры

Триггер — это механизм автоматического выполнения функции при наступлении события в таблице или базе данных. В таблицах триггеры обычно срабатывают на insert, update, delete или truncate. Они бывают before, after и instead of. Before выполняется до операции, after — после операции, instead of применяется в основном для представлений.

Триггеры используют для аудита, автоматического заполнения служебных полей, проверки сложных правил и синхронизации данных. Не стоит злоупотреблять триггерами для всей бизнес-логики, потому что скрытая логика усложняет сопровождение.

Пример триггера для автоматического обновления поля updated_at:

```
create table clients (  
    id bigserial primary key,  
    full_name text not null,  
    updated_at timestamp default now()  
);  
  
create function set_updated_at()  
returns trigger  
language plpgsql  
as $$  
begin  
    new.updated_at := now();  
    return new;  
end;  
$$;  
  
create trigger trg_clients_updated_at  
before update on clients  
for each row  
execute function set_updated_at();
```

Пример audit-триггера:

```
create table audit_log (  
    id bigserial primary key,  
    table_name text not null,  
    operation text not null,  
    changed_at timestamp default now()
```



```

);

create function log_client_changes()
returns trigger
language plpgsql
as $$
begin
    insert into audit_log (table_name, operation)
    values ('clients', tg_op);

    return coalesce(new, old);
end;
$$;

create trigger trg_clients_audit
after insert or update or delete on clients
for each row
execute function log_client_changes();

```

Триггеры событий срабатывают не на строки таблицы, а на DDL-события, например create table или drop table. Они применяются для аудита изменений структуры БД.

```

create function log_ddl_event()
returns event_trigger
language plpgsql
as $$
begin
    raise notice 'DDL operation: %', tg_tag;
end;
$$;

create event trigger trg_ddl_log
on ddl_command_end
execute function log_ddl_event();

```

23. Шифрование и хеширование данных

Шифрование и хеширование — разные механизмы защиты данных.

Шифрование обратимо: данные можно зашифровать, а затем расшифровать при наличии ключа. Хеширование необратимо: из хеша нельзя получить исходное значение нормальным способом. Поэтому пароли обычно не шифруют, а хешируют.

Для паролей нужно использовать специальные алгоритмы хеширования паролей, например bcrypt, а не обычные быстрые хеши вроде md5. Быстрые хеши плохо подходят для паролей, потому что их легко массово перебирать. В PostgreSQL для учебных примеров часто используют расширение pgcrypto.

```
create extension if not exists pgcrypto;

insert into users (login, password_hash)
values ('admin', crypt('secret_password', gen_salt('bf')));
```

Проверка пароля:

```
select id, login
from users
where login = 'admin'
and password_hash = crypt('secret_password', password_hash);
```

Шифрование применяют для конфиденциальных данных, если есть необходимость хранить значение в расшифровываемом виде. При этом главный вопрос — где хранить ключи. Если ключ лежит рядом с базой данных, защита становится слабее.

```
select pgp_sym_encrypt('secret text', 'encryption_key') as
encrypted_value;

select pgp_sym_decrypt(
    pgp_sym_encrypt('secret text', 'encryption_key'),
    'encryption_key'
) as decrypted_value;
```

Также важно защищать канал передачи данных через SSL/TLS, ограничивать доступ к серверу, не хранить пароли в открытом виде в коде и использовать минимально необходимые права.

24. Угрозы, специфичные для СУБД. Основные принципы безопасности. Персональные данные и конфиденциальность

К угрозам для СУБД относятся SQL-инъекции, утечка учётных данных, чрезмерные права пользователей, незащищённые резервные копии, ошибки в настройке сети, отсутствие журналирования, вредоносные действия администратора или пользователя, потеря данных из-за сбоя и несанкционированный доступ к персональным данным.

SQL-инъекция возникает, когда приложение склеивает SQL-запрос из пользовательского ввода. Правильное решение — параметризованные запросы, а не ручная конкатенация строк.

Плохой подход:

```
-- Пример опасной логики на уровне приложения:  
-- select * from users where login = ' + user_input + '
```

Правильный подход — передавать значения как параметры. В самой БД также можно использовать функции с параметрами.

```
create function find_user_by_login(p_login text)  
returns table (id bigint, login text)  
language sql  
as $$  
    select id, login  
    from users  
    where login = p_login;  
$$;
```

Основные принципы безопасности: минимальные права, разделение ролей, сложные пароли, параметризованные запросы, журналирование, резервное копирование, шифрование каналов связи, контроль доступа к резервным копиям, регулярное обновление системы и проверка настроек.

Персональные данные — это сведения, относящиеся к конкретному человеку: ФИО, телефон, адрес, паспортные данные, email, дата рождения и другие идентификаторы. Их нужно хранить только при наличии цели, ограничивать доступ, не выводить лишние поля в отчёты и по возможности маскировать данные.

Пример представления, скрывающего часть телефона:

```
create view public_clients as
select id,
       full_name,
       regexp_replace(phone, '(\+7)([0-9]{6})([0-9]{4})',
       '\1*****\3') as masked_phone
from clients;
```

25. Анализ данных средствами SQL: подготовка данных, выявление аномалий, уменьшение размерности

SQL можно использовать не только для хранения данных, но и для анализа. Первый этап анализа — подготовка данных: удаление дубликатов, обработка пропусков, приведение типов, нормализация форматов и проверка диапазонов. Часто исходные данные сначала загружают во временную или staging-таблицу, а затем очищают.

Пример поиска пропусков:

```
select *  
from clients  
where full_name is null  
or phone is null;
```

Пример поиска дубликатов:

```
select phone, count(*)  
from clients  
group by phone  
having count(*) > 1;
```

Аномалии — это значения, которые сильно отличаются от обычных. Их можно искать через минимумы и максимумы, z-score, межквартильный размах или процентильные границы. Простой пример — найти платежи, которые сильно выше среднего.

```
with stats as (  
    select avg(amount) as avg_amount,  
           stddev(amount) as std_amount  
    from payments  
)  
select p.*  
from payments as p  
cross join stats as s  
where p.amount > s.avg_amount + 3 * s.std_amount;
```

Уменьшение размерности в контексте SQL обычно означает сокращение числа признаков или агрегирование данных. Например, вместо всех платежей можно

построить таблицу признаков по клиентам: количество оплат, сумма оплат, дата последней оплаты.

```
select client_id,  
       count(*) as payments_count,  
       sum(amount) as total_amount,  
       max(paid_at) as last_payment_at  
from payments  
group by client_id;
```

Такие агрегированные наборы данных удобно использовать для отчётов, сегментации клиентов и дальнейшего анализа.

26. Анализ данных средствами SQL: временные ряды, когортный анализ, текстовый анализ, анализ экспериментов

Временной ряд — это данные, упорядоченные по времени. В SQL временные ряды часто анализируют через группировку по дням, месяцам или неделям. Для этого удобно использовать `date_trunc`.

```
select date_trunc('month', paid_at)::date as month,
       sum(amount) as revenue
from payments
group by date_trunc('month', paid_at)::date
order by month;
```

Для сравнения с предыдущим периодом применяют оконную функцию `lag`.

```
with monthly as (
  select date_trunc('month', paid_at)::date as month,
         sum(amount) as revenue
  from payments
  group by date_trunc('month', paid_at)::date
)
select month,
       revenue,
       revenue - lag(revenue) over (order by month) as diff
from monthly;
```

Когортный анализ группирует пользователей по первому событию, например по месяцу регистрации, и отслеживает их поведение в следующие периоды.

```
with first_order as (
  select client_id,
         date_trunc('month', min(created_at))::date as
cohort_month
  from orders
  group by client_id
), activity as (
  select o.client_id,
         date_trunc('month', o.created_at)::date as
activity_month
  from orders as o
)
select f.cohort_month,
```

```
        a.activity_month,  
        count(distinct a.client_id) as active_clients  
from first_order as f  
join activity as a on a.client_id = f.client_id  
group by f.cohort_month, a.activity_month  
order by f.cohort_month, a.activity_month;
```

Текстовый анализ может включать поиск слов, регулярные выражения, подсчёт длины текста и полнотекстовый поиск. Для простого поиска используют like, ilike или регулярные выражения.

```
select *  
from reviews  
where comment ilike '%хорошо%';
```

Анализ экспериментов обычно сравнивает группы А и В по метрике: средний чек, конверсия, количество действий.

```
select experiment_group,  
       count(*) as users_count,  
       avg(order_amount) as avg_order_amount  
from experiment_results  
group by experiment_group;
```


27. Создание сложных наборов данных. Общие табличные выражения. Расширения для группировки

Сложные наборы данных часто создаются в несколько шагов. Для этого удобно использовать общие табличные выражения, или СТЕ. Они задаются через `with` и позволяют разбить большой запрос на понятные блоки. СТЕ особенно полезны для отчётов и аналитических выборок.

```
with client_totals as (  
    select client_id,  
           sum(amount) as total_amount  
    from payments  
    group by client_id  
) , client_orders as (  
    select client_id,  
           count(*) as orders_count  
    from orders  
    group by client_id  
)  
select c.full_name,  
       coalesce(t.total_amount, 0) as total_amount,  
       coalesce(o.orders_count, 0) as orders_count  
from clients as c  
left join client_totals as t on t.client_id = c.id  
left join client_orders as o on o.client_id = c.id;
```

СТЕ делают запрос читаемее, но не нужно превращать простой запрос в слишком много промежуточных блоков. Если запрос можно понятно написать через обычный `join` или `group by`, лучше не усложнять.

PostgreSQL поддерживает расширения группировки: `grouping sets`, `rollup` и `cube`. Они позволяют получать несколько уровней агрегирования одним запросом.

`rollup` строит итоги по иерархии. Например, можно получить суммы по году, месяцу и общий итог.

```
select extract(year from paid_at) as year,  
       extract(month from paid_at) as month,  
       sum(amount) as total_amount  
from payments  
group by rollup (  
    extract(year from paid_at),
```

```
        extract(month from paid_at)
    )
    order by year, month;
```

cube строит агрегаты по всем комбинациям измерений.

```
select payment_method,
       status,
       sum(amount) as total_amount
from payments
group by cube (payment_method, status);
```

Такие конструкции удобны для отчётов, где нужны промежуточные и общие итоги.

28. Администрирование базы данных: конфигурирование сервера, архитектура, организация данных

Администрирование базы данных включает установку сервера, настройку конфигурации, управление пользователями, контроль производительности, резервное копирование, восстановление и мониторинг. В PostgreSQL основные настройки находятся в файлах `postgresql.conf`, `pg_hba.conf` и `pg_ident.conf`.

`postgresql.conf` отвечает за параметры сервера: порт, память, журналирование, автовакуум, настройки WAL, планировщик и другие параметры. `pg_hba.conf` управляет правилами подключения: кто, откуда и каким способом аутентификации может подключаться. `pg_ident.conf` используется для сопоставления внешних пользователей с ролями PostgreSQL.

Архитектура PostgreSQL включает клиентские процессы, главный серверный процесс, отдельные backend-процессы для подключений, shared buffers, WAL, фоновые процессы, автовакуум и writer-процессы. Когда клиент выполняет запрос, PostgreSQL разбирает SQL, строит план, обращается к данным и возвращает результат.

Организация данных строится вокруг кластера баз данных. Кластер содержит базы данных, базы содержат схемы, схемы содержат таблицы, индексы, функции и другие объекты. Физически данные хранятся в каталоге `data directory`, но вручную изменять файлы данных нельзя.

Пример просмотра активных подключений:

```
select pid,  
       username,  
       datname,  
       state,  
       query  
from pg_stat_activity;
```

Пример просмотра размера базы данных:

```
select pg_database_size(current_database()) as size_bytes,  
       pg_size_pretty(pg_database_size(current_database())) as  
size_pretty;
```

Администратор должен следить за доступами, свободным местом, резервными копиями, журналами, длительными запросами и блокировками.

29. Администрирование базы данных: резервное копирование. Репликация

Резервное копирование нужно для восстановления данных после ошибки пользователя, сбоя диска, повреждения базы или неудачного обновления. В PostgreSQL часто используют логическое и физическое резервное копирование.

Логическое резервное копирование выполняется через `pg_dump`. Оно создаёт SQL-файл или архив с командами для восстановления объектов и данных. Такой способ удобен для переноса базы между серверами и для резервных копий отдельных баз.

```
-- В терминале, не внутри SQL:  
-- pg_dump -U postgres -d driving_school -F c -f  
driving_school.backup
```

Восстановление архива выполняют через `pg_restore`.

```
-- В терминале:  
-- pg_restore -U postgres -d driving_school_restored  
driving_school.backup
```

Если `dump` сделан в обычный SQL-файл, его можно восстановить через `psql`.

```
-- В терминале:  
-- psql -U postgres -d driving_school_restored -f backup.sql
```

Физическое резервное копирование копирует файлы кластера и WAL-журналы. Оно полезно для больших баз и восстановления на конкретный момент времени. Для этого используют `pg_basebackup` и архивацию WAL.

Репликация — это механизм копирования изменений с основного сервера на один или несколько дополнительных серверов. Она повышает отказоустойчивость и может разгружать чтение. В PostgreSQL распространена потоковая репликация, где `standby`-сервер получает WAL-записи от `primary`-сервера.

Репликация не заменяет резервное копирование. Если пользователь случайно удалил важные данные, ошибка может быстро попасть и на реплику. Поэтому для защиты нужны и реплики, и резервные копии.

30. Администрирование базы данных: мониторинг сервера БД, журналирование, блокировки, задачи администрирования

Мониторинг нужен, чтобы понимать состояние сервера и вовремя замечать проблемы. В PostgreSQL можно отслеживать активные подключения, длительные запросы, блокировки, размеры таблиц, использование индексов, ошибки в журналах и работу autovacuum.

Активные запросы смотрят через pg_stat_activity.

```
select pid,
       username,
       state,
       now() - query_start as query_time,
       query
from pg_stat_activity
where state <> 'idle'
order by query_time desc;
```

Блокировки возникают, когда несколько транзакций пытаются одновременно работать с одними и теми же объектами. Не все блокировки плохие: они нужны для целостности. Проблемой становятся долгие блокировки, из-за которых другие запросы ждут.

```
select locktype,
       relation::regclass as relation_name,
       mode,
       granted,
       pid
from pg_locks
where relation is not null;
```

Журналирование настраивается в postgresql.conf. В журнал можно записывать ошибки, подключения, отключения, длительные запросы и другие события. Для поиска медленных запросов полезен параметр log_min_duration_statement.

Размеры таблиц и индексов можно проверять так:

```
select relname as table_name,
       pg_size_pretty(pg_total_relation_size(relid)) as
total_size
```

```
from pg_catalog.pg_statio_user_tables  
order by pg_total_relation_size(relid) desc;
```

К задачам администрирования относятся создание пользователей, выдача прав, резервное копирование, восстановление, настройка конфигурации, обновление сервера, анализ производительности, контроль журналов, vacuum, analyze, reindex при необходимости и проверка свободного места.

```
vacuum analyze clients;
```

Хорошее администрирование — это не только исправление проблем, но и профилактика: регулярные бэкапы, проверка восстановления, мониторинг диска, ограничение прав и анализ медленных запросов.